

Sending GET Requests

 <https://github.com/learn-co-curriculum/python-p4-sending-get-requests>  <https://github.com/learn-co-curriculum/python-p4-sending-get-requests/issues/new>

Learning Goals

- Create a request to a remote resource.
 - Handle the response.
 - Learn how to parse JSON.
-

Key Vocab

- **Request:** an attempt by one machine to contact another over the internet.
 - **Client:** an application or machine that accesses services being provided by a server through the internet.
 - **Web Server:** a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.
-

Introduction

Simple Python applications are capable of powerful feats. We can use them to handle all sorts of routine tasks on our computer. We can use them to represent real-world relationships and systems. It's like we've been given an infinite multi-tool that can help us work all kinds of data and make it useful.

The limitation at this point, it seems, is really that we don't have enough data to play with. Luckily, we have a massive source of data available to us: the Internet!

We typically experience the Internet through webpages in a browser, but a lot of information is accessible as pure data. If we know the tools to retrieve and organize that data, we'll have access to a wealth of knowledge. In this lesson, we're going to introduce some of the basic ways in which we can get remote data using Python.

Making an HTTP Request through the Internet

When we go to a website in the browser, we type in a URL and press enter. When enter is pressed, the browser sends a request to that URL. The request travels to a server somewhere in the world that is hosting the website we want. The server sends back a response. This response includes the HTML code structure of the website, which your browser uses to build the page for you in front of your eyes.

This type of request is known as a HTTP GET request. We are simply retrieving information at a specified location. There are [other types of requests](#)  https://www.w3schools.com/tags/ref_httpmethods.asp for sending, updating and deleting information that we explore in later lessons, but for now, we will focus on the GET request. Don't worry though, the GET request can... *get* us quite far on its own!

In Python code, we don't have the luxurious graphical user interface of a browser, but we can still send GET requests in a similar fashion using built-in Python modules and classes.

Sending an HTTP GET Request Using Python

To show how to make requests in Python, we'll first make an HTTP GET request in repl. To start, once repl is open, we need to import [requests](#)  <https://requests.readthedocs.io/en/latest/>:

```
import requests
```

If loaded correctly, we will set a URL that we want to get data from. For this example, we'll get data from a GitHub repository that is hosting a webpage: <https://learn-co-curriculum.github.io/json-site-example/>. We can store this URL in a variable:

```
url = "https://learn-co-curriculum.github.io/json-site-example/"  
#=> "https://learn-co-curriculum.github.io/json-site-example/"
```

The first thing we do is pass this `url` variable into a method called `get` that is part of the requests module we loaded with `import 'requests'` :

```
response = requests.get(url)
```

Aside: [URI](https://en.wikipedia.org/wiki/Uniform_Resource_Identifier)  (https://en.wikipedia.org/wiki/Uniform_Resource_Identifier) stands for Universal Resource Identifier. URIs, in short, are a standard way to name a resource. [URL](https://en.wikipedia.org/wiki/URL)  (<https://en.wikipedia.org/wiki/URL>) stands for Universal Resource Locator. They are a standard way to *locate* something. **URLs also happen to act as a standard name** (a web address acts as both the name of the website *and* the text you need to enter to visit the website). Therefore, generally speaking, all URLs are considered a subset of URIs (but not all URIs are URLs).

Reading the Body of a Response

The `requests.get` function will return a Response object. We can get the HTML as a unicode string by using `response.text`

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <meta http-equiv="X-UA-Compatible" content="ie=edge">  
  <title>JSON Example Site</title>  
</head>  
<body>  
  <h1>JSON Example Site</h1>  
  
  <h3>Endpoints available to visit:</h3>
```

```
<ul>
  <li><a href="https://learn-co-curriculum.github.io/json-site-example/endpoints/locations.json" alt="locations JSON">locations JSON</a>
  <li><a href="https://learn-co-curriculum.github.io/json-site-example/endpoints/people.json" alt="people JSON">people JSON</a>
</ul>
</body>
</html>
```

We requested and received a *webpage*. it is a great start! And just to review, we did this using only a couple lines of Python:

```
import requests
url = "https://learn-co-curriculum.github.io/json-site-example/"
response = requests.get(url)
print(response.text)
```

So, we've seen how to get the HTML code from a webpage. Although this is cool, in its current state, this isn't very useful information. There are actually tools designed specifically to take this raw HTML and turn it into organized data for us in a process known as scraping, which we learned about in the previous optional module. Plenty of data is already organized and made available to us separate from HTML code and we can retrieve it the same way we send a GET request to a website.

Defining JSON

JSON stands for JavaScript Object Notation and is a standard way store and transfer nested data over the internet. The keyword here is **Notation**. Data stored in JSON format is actually just data stored *as a string*, but structured in a way that is easily converted into usable nested data. Python has a built-in **JSON** [↗\(https://docs.python.org/3/library/json.html\)](https://docs.python.org/3/library/json.html) module that includes a **loads** method to take JSON formatted data and turn it into a list.

Retrieving JSON Data

Just as with the website, we write in the *Universal Resource Locator*, convert it to a **URI** object, then send a GET request. Previously, we sent requests to <https://learn-co-curriculum.github.io/json-site-example/>. The server, in this case, had an HTML file to serve in response. If we request a slightly different URL, we'll get a different response. This site has some example data stored as JSON on it. By changing out the request to <https://learn-co-curriculum.github.io/json-site-example/endpoints/locations.json> → <https://learn-co-curriculum.github.io/json-site-example/endpoints/locations.json>), instead of responding with HTML, this time, the server will send the JSON data.

```
import requests
url = "https://learn-co-curriculum.github.io/json-site-example/endpoints/locations.json"

response = requests.get(url)

print(response.content)
```

Running the above in a fresh repl session, you should see that the **response.content** now returns a byte string:

```
b'[\n  {\n    "name": "Flatiron School Manhattan",\n    "address": "11 Broadway, New York, NY 10004",\n    "coordinate
```

This is JSON! It isn't very friendly to use yet, so we'll want to convert it. First, we'll need to require the **json** module:

```
import json
```

Then we pass in **response.body** to the JSON parser:

```
json.loads(response.content)
```

The result is an list containing five dictionaries, each with some data about a Flatiron School campus:

```
[
  {
    "name": "Flatiron School Manhattan",
    "address": "11 Broadway, New York, NY 10004",
    "coordinates": {
      "latitude": "40.704521",
      "longitude": "-74.012833"
    }
  },
  {
    "name": "Flatiron School Austin",
    "address": "316 W 12th St, Austin, TX 78701",
    "coordinates": {
      "latitude": "30.275080",
      "longitude": "-97.743700"
    }
  },
  {
    "name": "Flatiron School Denver",
    "address": "3601 Walnut St 5th Floor, Denver, CO 80205",
    "coordinates": {
      "latitude": "39.743510",
      "longitude": "-105.011360"
    }
  },
  {
    "name": "Flatiron School Seattle",
    "address": "1411 4th Ave 13th Floor, Seattle, WA 98101",
    "coordinates": {
      "latitude": "47.684879",
      "longitude": "-122.201363"
    }
  }
]
```

```
{,
{
  "name": "Flatiron School London",
  "address": "131 Finsbury Pavement, Finsbury, London EC2A 1NT, UK",
  "coordinates": {
    "latitude": "51.520480",
    "longitude": "-0.087190"
  }
}
```

Aside: If you'd like a better view of this data, You can use the json `dumps` method. The `indent = 4` argument will format the json in a more readable manner.

```
json_content = json.loads(response.text)
print(json.dumps(json_content, indent=4, sort_keys=True))
```

The completed code for this example is in the `get.py` file. Try updating this code to make requests to other JSON APIs by changing the URL!

Conclusion

If you've been following along, you've made three HTTP GET requests! We can make these requests using a URL and some Python libraries. Some data is readily available as JSON, which we can retrieve and convert into Python data structures.

Having these tools unlocks access to *a lot of data*. Being able to communicate with remote resources is also the cornerstone of web development!

Resources

- [GET - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET) ➞ [\(https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)
- [HTTP methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) ➞ [\(https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)
- [requests](https://requests.readthedocs.io/en/latest/) ➞ [\(https://requests.readthedocs.io/en/latest/\)](https://requests.readthedocs.io/en/latest/)
- [Python JSON](https://docs.python.org/3/library/json.html) ➞ [\(https://docs.python.org/3/library/json.html\)](https://docs.python.org/3/library/json.html)